

HASHDIT

HashDit

Smart Contract Security

Venus Liquidity Hub Receipt Tracking

Security Audit Report

Venus

3 Nov 2026 – 6 Nov 2026

This report is provided on an "as is" basis and does not constitute investment advice, a recommendation, or an endorsement of the audited project. HashDit makes no representations or warranties regarding the safety, functionality, or correctness of the smart contracts beyond the scope defined herein.

The audit was performed based on the source code provided at the time of the engagement. Any subsequent modifications to the codebase may invalidate the findings of this report. HashDit shall not be held liable for any losses or damages resulting from the use, deployment, or interaction with the audited contracts.

The scope of this audit is limited to the smart contracts listed in this report. It does not cover off-chain components, front-end interfaces, or third-party integrations unless explicitly stated. Users and developers should conduct their own due diligence before interacting with or deploying the audited contracts.

Audit Period	3 Nov 2026 - 6 Nov 2026
Overall Risk	**Low**
Project Name	Venus Liquidity Hub Receipt Tracking
Github	https://github.com/VenusProtocol/venus-liquidity-hub/pull/40
Commit	c7ee4e183fae02c3c98f4cc2a695661d1b0d54d1

Smart Contract List

No.	Contract Name	Verdict	Details
1	AdapterCoreV1.sol	Low	[L01]-Ack
2	YieldGroupBase.sol	Low	[L01]-Ack [L02]-Fixed [I01]-Ack [I02]-Fixed
3	Hub.sol	Low	[L01]-Ack
4	AdapterSpokeV1.sol	Low	[L02]-Fixed
5	YieldGroupBaseStorage.sol	Informational	[I01]-Ack

Findings

[L01] Core maxWithdraw always deducts full reserves and ignores newly accrued reserves, so MAX withdrawals can still revert	
Contract	adapters/AdapterCoreV1.sol, base/YieldGroupBase.sol, Hub.sol
Severity	Low

Description	This PR changes the cash bound in AdapterCoreV1.maxWithdraw to getCash() − totalReserves(), read from stored state:
	
	JavaScript
	`uint256 cash = IVToken(resource).getCash();`
	`uint256 reserves = IVToken(resource).totalReserves();`
	`if (cash <= reserves) return 0;`
	`cash -= reserves;`
	`...`
	`uint256 redeemableGross = ourValue < cash ? ourValue : cash;`
	
	This has two problems:
	1) Venus Core vToken's redeemFresh only checks getCashPrior(). Reserves that have not been swept are still redeemable; they leave cash only when an accrual reaches the reduceReservesBlockDelta interval. The PR deducts all reserves whether or not a sweep is due, which is stricter than the vToken itself.
	2) Every accrueInterest() books new reserves ΔR (interestAccumulated × reserveFactorMantissa), and the quote does not account for them.
	Hub.withdraw / redeem accrue first, then recompute maxWithdraw(owner) on the new state as the ERC-4626 limit. When the quote is cash-bound, the enforced limit is ΔR below the quote, so the call reverts with ERC4626: withdraw more than max. Before the fix, reverts only happened in the sweep block. After the fix, they also happen in blocks where no sweep is due, which used to succeed.
	
	The project's fork test already observes this gap (it asserts quoted − requoted <= 1e18), but it never asserts that withdraw(quoted) succeeds.
	
	Impact
	When the Hub's position in a Core market exceeds that market's payable cash, withdrawing or redeeming the full quoted amount fails, and users have to lower the amount by hand. No funds are at risk. On mainnet, vUSDT's payable cash is about 6.9× the Hub's position, so the issue does not trigger today.

Recommendation	Simulate the current block's accrual in maxWithdraw. Deduct reserves, including the ΔR estimated up to the current block, only when block.number − reduceReservesBlockNumber ≥ reduceReservesBlockDelta. Otherwise, quote against getCash() to match the vToken's redeemFresh. Note: a proportional margin inside maxWithdraw does not help, because the Hub enforces the limit with the same function and the margin cancels out.
Status	Acknowledged. The project argues that Core's maxWithdraw deducting against current totalReserves (rather than estimating the reserves the next accrueInterest would add) is a negligible gap — the Hub's position in the market is well below available cash, and any single user's withdraw is capped by their own balance regardless of reserves. No code was changed (AdapterCoreV1.sol:209-233 still does cash -= reserves)

[L02] Donated dust can block removal of a Spoke resource	
Contract	AdapterSpokeV1.sol, YieldGroupBase.sol
Severity	Low

Description	This PR adds a receipt ledger, _trackedReceipt. The YieldGroup now reads the ledger, not the wallet, for valuation, liquidity, cap room, resource removal and the withdraw-queue check. Receipts donated straight to the wallet never enter the ledger.
	
	AdapterSpokeV1.withdraw is a stateless adapter executed via delegatecall, so it cannot see the ledger. It contains a whole-balance redeem that clears zero-value remainders, and that check still reads the wallet balance:
	
	JavaScript
	`uint256 tokens = _burnFor(exchangeRate, amount);`
	`uint256 vBal =`
	`IVTokenIsolated(resource).balanceOf(address(this)); // wallet`
	`balance, donations included`
	`if (tokens < vBal && ((vBal - tokens) * exchangeRate) /`
	`EXP_SCALE == 0) {`
	`uint256 payableCash = getCash() - totalReserves();`
	`if ((vBal * exchangeRate) / EXP_SCALE <= payableCash)`
	`tokens = vBal;`
	`}`

Before the final withdrawal, an attacker sends a few vTokens to the YieldGroup so that the wallet remainder vBal - tokens is worth at least 1 wei. The whole-balance redeem then does not trigger, and only tokens are burned. The ledger remainder tracked - tokens, however, is worth 0, which causes three things:

- _liquidAssetValue reports 0, so the remainder can never be withdrawn.
- removeResource reverts with ResourceHasBalance because the ledger is non-zero.
- The resource cannot be dropped from the inner withdraw queue either.

The PR handles the case where the whole-balance redeem burns donated units along with tracked ones (the ledger is floored at 0). It does not handle the opposite case, where a donation stops the whole-balance redeem from triggering. The second condition, (vBal * exchangeRate) / EXP_SCALE <= payableCash, also reads the wallet balance, so a larger donation can stop the whole-balance redeem from triggering in the same way.

Impact

On a Spoke market whose exchange rate is below 1e18, an attacker can stop governance from removing the resource for about 1 wei plus gas. No funds are lost. The only recovery is a governance VIP that calls syncReceipt, withdraws again, then removes the resource. The currently deployed USDT / USDC / U Spoke markets have exchange rates around 1e28 and are not affected, but the adapter explicitly supports markets below 1e18.

Recommendation

When YieldGroupBase._dispatchWithdraw updates the ledger, clear any remainder that is worth zero:

```javascript
if (left != 0 && adapter.receiptValue(resource, left) == 0)
left = 0;
```

This puts dust cleanup in the YieldGroup, which owns the ledger. The adapter interface does not change, and NAV does not move. After the resource is removed, the leftover donation and dust in the wallet can be recovered with sweep. Add a test: on a Spoke market with an exchange rate below 1e18, donate a few vTokens before the final withdrawal and assert that the resource can still be removed.

Status

Fixed in commit 71b2729. _dispatchWithdraw now zeroes out any leftover tracked amount that is worth nothing, so donated dust can no longer block removeResource:

```javascript
uint256 left = burned < tracked ? tracked - burned : 0;
if (left != 0 &&
IResourceAdapter(adapter).receiptValue(resource, left) == 0)
left = 0;
_trackedReceipt[resource] = left;
```

[I01] Upgrade must be atomic, or NAV is understated until the ledger is synced	
Contract	base/YieldGroupBaseStorage.sol base/YieldGroupBase.sol
Severity	Informational
Description	The _trackedReceipt mapping added by this PR is a new storage slot. After the upgrade, the ledger for every existing resource starts at 0, while YieldGroup valuation, liquidity, deposit room and removal checks all read from the ledger. The upgrade must therefore complete three steps in order: Upgrade the YieldGroup beacon. 1. Call updateResourceAdapter on every tracked resource to switch to the new adapter that implements receiptValue. 2. Call syncReceipt on every tracked resource to align the ledger with the wallet balance. If only part of this is done: 1) Beacon upgraded only: the new code calls receiptValue on the old adapter, which does not have it. The group's totalAssets() reverts, and all Hub deposits, withdrawals and redemptions fail. 2) Adapters switched but not yet synced: the ledger is 0, so the group's NAV is only its idle balance and the Hub share price is heavily understated. Anyone can deposit cheaply and redeem at full value once the sync lands, at the expense of existing LPs. 3) A resource is missed by the sync: that resource stays valued at 0, and removeResource, the withdraw-queue check and Hub.removeYieldGroup all treat it as holding no funds. In addition, updateResourceAdapter re-runs validateRegistration against the new adapter. If a Core market has a non-zero treasuryPercent, or a Spoke market has been delisted or has enabled an allowlist that excludes the YieldGroup, the entire upgrade VIP reverts. **Impact** If the upgrade is not atomic or a resource is missed, NAV is understated for a window in which it can be arbitraged, causing losses to existing LPs. The

	project's fork tests cover all three stages, and an atomic VIP never exposes this window.
Recommendation	1. Execute the following in a single atomic VIP: grant syncReceipt(address), upgrade the beacon, switch adapters, and call syncReceipt on every tracked resource. Confirm in the VIP simulation that no resource is missed. 2. Re-simulate on a fork at the execution block before execution to confirm every resource still passes validateRegistration.
Status	Acknowledged. The upgrade ships as a single VIP (grants syncReceipt, flips the beacon, switches every adapter, syncs every resource in one shot), so a failure in any step reverts the whole VIP, and the team plans to rehearse it on a fork before execution to confirm no resource is missed. This is a deployment-process guarantee, not a code change, and the reasoning is sound.

[I02] syncReceipt syncs to the wallet balance at execution time and may admit externally transferred receipts	
Contract	base/YieldGroupBase.sol
Severity	Informational

Description	syncReceipt sets _trackedReceipt[resource] to whatever adapter.receiptBalance(resource, address(this)) returns at execution time. There is no parameter for the expected amount:
	
	JavaScript
	`uint256 held = IResourceAdapter(cfg.adapter).receiptBalance(resource, address(this));`
	`_trackedReceipt[resource] = held;`
	
	Governance evaluates the balance when the proposal is made, but the sync admits the balance at execution. Between proposal and execution, anyone can transfer the same receipt token to the YieldGroup, and those receipts are added to the ledger by the sync. The ledger can only move up, so admitted receipts cannot be removed, and sweep cannot recover a registered resource.
	
	Receipts from a healthy market are simply a gift to LPs and cause no harm. If the resource's exchange rate is overstated, however (for example, a Spoke market with bad debt that has not been socialized), the sync reopens the donation attack this PR is meant to close. An attacker holds Hub shares in advance, transfers overvalued vTokens before the sync, and after the sync redeems from healthy resources at the inflated NAV.

In addition, the sync causes a one-off jump in NAV. Since VIP execution times are public, anyone can deposit beforehand and capture part of the donation. The jump is also visible to the performance fee: Hub._accrueFees compares the current PPS against the high-water mark and charges performanceFeeBps on any increase, so on the next accrual after a sync, part of the admitted receipts is minted to the fee recipient as if it were yield, rather than accruing to LPs.

Impact

Exploitation requires governance to sync a resource whose exchange rate is overstated. On mainnet, existing governance functions are granted only to the Normal Timelock, and syncReceipt is expected to be called only through a VIP. Governance therefore has time to review the resource before execution and cancel the proposal if needed, so the practical risk is low. The performance-fee effect applies only when performanceFeeBps is non-zero. Fees launch at 0/0, so it does not apply today.

Recommendation

- Add an expected-amount parameter to syncReceipt and revert if the actual balance differs at execution.
- Check the resource for bad debt before syncing. If a donation does need to be admitted, temporarily raise redeemFeeBps beforehand.
- If a performance fee is active, set performanceFeeBps to 0 in the same VIP, then sync and accrue so the high-water mark moves to the post-sync PPS, then restore the fee.

Status

Fixed in commit fe8d150. syncReceipt now takes a maxAdded cap, limiting how much can be admitted into the tracked balance in one sync to what governance reviewed:

JavaScript

```
function syncReceipt(address resource, uint256 maxAdded)
external override {
    ...
    uint256 held =
IResourceAdapter(cfg.adapter).receiptBalance(resource,
address(this));
    uint256 old = _trackedReceipt[resource];
    uint256 synced = held;
    if (held > old && held - old > maxAdded) synced = old +
maxAdded;
    _trackedReceipt[resource] = synced;
}
```